JOBSHEET 14

IMPLEMENTASI LOGIN DENGAN NEXTAUTH

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_14

C. Langkah Praktikum

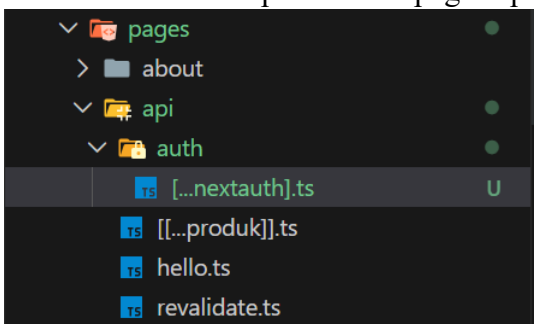
Bagian 1 – Install NextAuth

- npm install next-auth –force

```
PS D:\Pemograman_Framework\praktikum_14\my-app-14> npm install next-auth --force
npm WARN using --force Recommended protections disabled.

added 15 packages, and audited 447 packages in 13s

148 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

Bagian 2 – Konfigurasi API Auth

- Buat file dan folder pada folder pages/api/auth/[...nextauth].ts



- o Modifikasi file [...nextauth].ts:

```
1  import NextAuth, { NextAuthOptions } from "next-auth";
2  import CredentialsProvider from "next-auth/providers/credentials";
3
4  export const authOptions: NextAuthOptions = {
5    session: {
6      strategy: "jwt"
7    },
8    secret: process.env.NEXTAUTH_SECRET,
9    providers: [
10     CredentialsProvider({
11       name: "credentials",
12       credentials: {
13         fullname: { label: "Full Name", type: "text" },
14         email: { label: "Email", type: "email" },
15         password: { label: "Password", type: "password" }
16       },
17       async authorize(credentials) {
18         const user: any = {
19           id: "1",
20           email: credentials?.email,
21           password: credentials?.password,
22           fullname: credentials?.fullname
23         };
24         if (user) {
25           return user;
26         } else {
27           return null;
28         }
29       }
30     })
31   ],
32   callbacks: {
33     async jwt({ token, account, profile, user }: any) {
34       if (account?.provider === "credentials" && user) {
35         token.email = user.email;
36       }
37       return token;
38     },
39     async session({ session, token }: any) {
40       if (token.email) {
41         session.user.email = token.email;
42       }
43       return session;
44     }
45   }
46 };
47
48 export default NextAuth(authOptions);
49
```

Bagian 3 – Tambahkan Secret

- Buka file .env.local dan tambahkan code pada line 12
- NEXTAUTH_SECRET=RANDOM_BASE64_STRING
- Untuk mendapatkan nilai RANDOM_BASE64_STRING gunakan generator RANDOM_BASE64_STRING seperti https://www.convertsimple.com/random-base64-generator/

```
[...nextauth].ts    .env.local
my-app-14 > .env.local
9
10
11  # gunakan website https://generate-secre
12  # lalu masukkan ke dalam variabel NEXTAU
13  NEXTAUTH_SECRET=dG9vbGV2ZW50bW9yZWxpa2V0
```

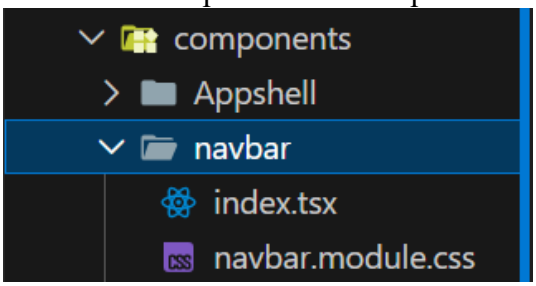
Bagian 4 – Tambahkan SessionProvider

- Buka file _app.tsx dan modifikasi:

```
1  import '@/styles/globals.css'
2  import type { AppProps } from 'next/app'
3  import AppShell from '@/components/Appshell'
4  import Navbar from '@/components/index'
5  import { Session } from 'node:inspector'
6  import { SessionProvider } from 'next-auth/react'
7
8  export default function App({ Component, pageProps: { session, ...pageProps } }: AppProps) {
9    return (
10     <SessionProvider session={pageProps.session}>
11     <AppShell>
12       <Component {...pageProps} />
13     </AppShell>
14     </SessionProvider>
15   );
16 }
17
```

Bagian 5 – Tambahkan Tombol Login & Logout

- Buka index.tsx pada folder component/navbar:



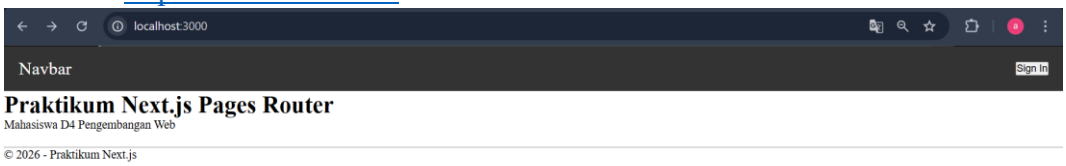
- Modifikasi file index.tsx pada line 10 dan 2

```
1  import syles from './navbar.module.css';
2  import { signIn, signOut, useSession } from 'next-auth/react';
3
4  const Navbar = () => {
5      // const { data: session } = useSession();
6
7      return (
8      <div className={syles.navbar}>
9          <div className="big">
10             Navbar
11     </div>
12     <button onClick={() => signIn()}>Sign In</button>
13     </div>
14     );
15 };
16
17 export default Navbar;
```

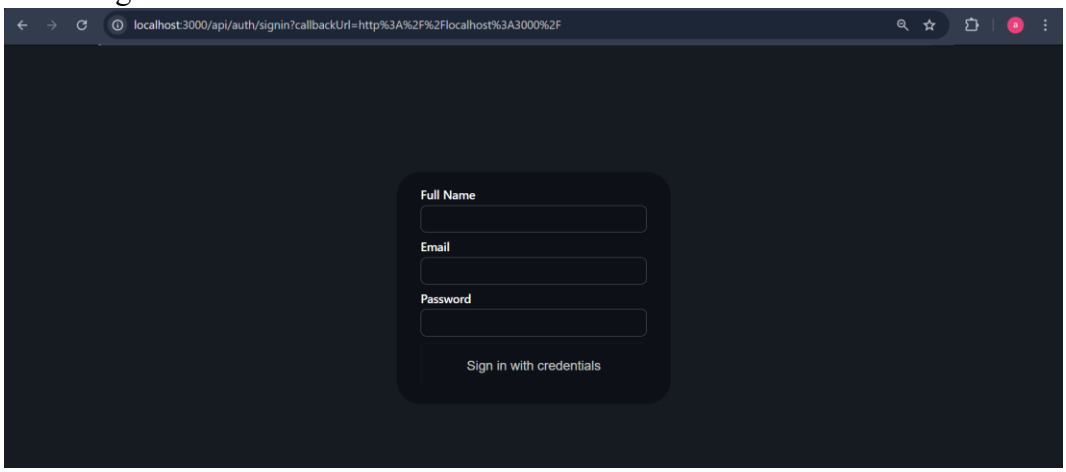
- Buka file file navbar.module.scss tambahkan code pada line 9

```
1  .navbar {
2    width: 100%;
3    height: 60px;
4    background-color: #333;
5    color: white;
6    display: flex;
7    align-items: center;
8    padding: 0 20px;
9    justify-content: space-between;
10 }
```

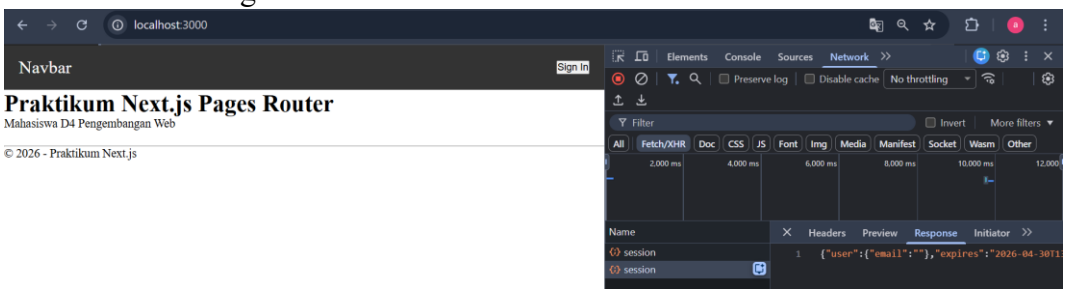
- Jalankan http://localhost:3000/



- Jika di klik sign in maka akan muncul dan isikan textbox masing. Setelah itu klik button sign in dan setelah diklik maka akan kembali ke halaman localhost



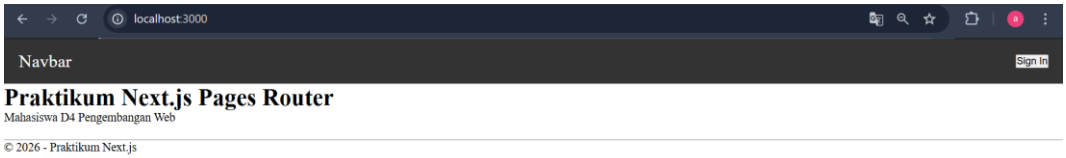
- Setelah berhasil login maka akan muncul session



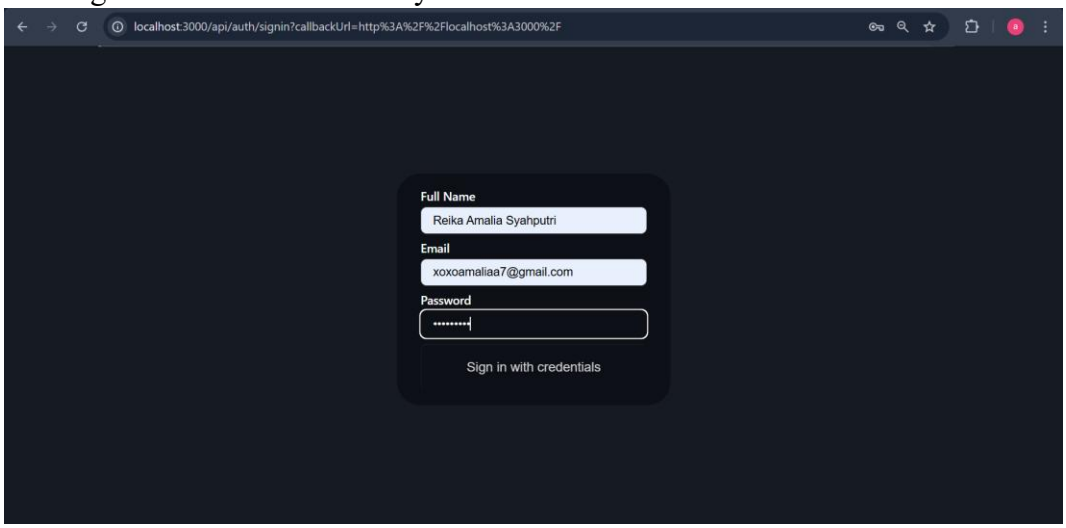
- Untuk dapat menangkap data pada session maka tambahkan code sebagai berikut :

```
import syles from './navbar.module.css';
import { signIn, signOut, useSession } from 'next-auth/react';

const Navbar = () => {
    const { data } = useSession();
    // const { data: session } = useSession();

    return (
    <div className={syles.navbar}>
        <div className="big">
            Navbar
    </div>
    {data ? (
    <button onClick={() => signIn()}>Sign In</button>
    ) : (
    <button onClick={() => signOut()}>Sign Out</button>
    )}
    </div>
    );
};

export default Navbar;
```

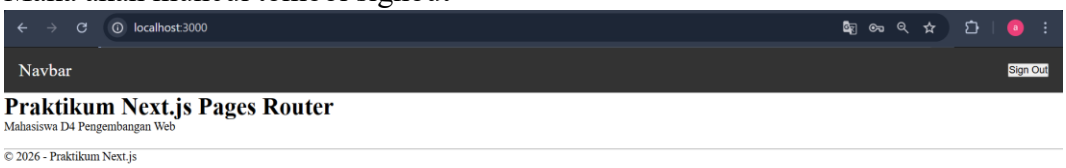
- Uji coba sign in dan sign out
  Jalankan Kembali npm run dev
  Jalankan localhost



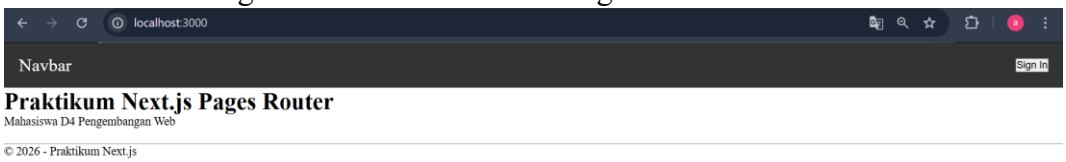
- Klik sign in dan isikan textboxnya



- Maka akan muncul tombol signout



- Ketika user klik signout maka akan kembali sign in



D. Menambahkan Data Tambahan (Full Name)

- Buka file [...nextauth].js dan tambahkan code pada line 22

```
import NextAuth, { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt"
  },
  secret: process.env.NEXTAUTH_SECRET,
  providers: [
    CredentialsProvider({
      name: "credentials",
      credentials: {
        fullname: { label: "Full Name", type: "text" },
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        const user: any = {
          id: "1",
          email: credentials?.email,
          password: credentials?.password,
          fullname: credentials?.fullname,
        };
        if (user) {
          return user;
        } else {
          return null;
        }
      }
    })
  ],

```

- Pada callbacks modifikasi codenya menjadi berikut :

```
callbacks: {
    async jwt({ token, account, profile, user }: any) {
      if (account?.provider === "credentials" && user) {
        token.email = user.email;
        token.fullname = user.fullname;
      }
      return token;
    },
    async session({ session, token }: any) {
      if (token.email) {
        session.user.email = token.email;
      }
      if (token.fullname) {
        session.user.fullname = token.fullname;
      }
      return session;
    }
  }
};

export default NextAuth(authOptions);

```

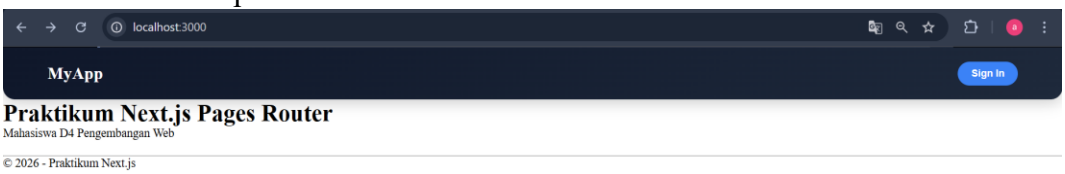
- Modifikasi navbar.module.scss

```scss
.navbar {
  width: 100%;
  height: 70px;
  padding: 0 60px;

  display: flex;
  align-items: center;
  justify-content: space-between;

  background: linear-gradient(135deg, #0f172a, #1e293b);
  color: #ffffff;

  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.2);
  border-bottom-left-radius: 16px;
  border-bottom-right-radius: 16px;
}

/* Logo / Brand */
.navbar__brand {
  font-size: 22px;
  font-weight: 700;
  letter-spacing: 0.5px;
}

/* Right Section */
.navbar__right {
  display: flex;
  align-items: center;
  gap: 32px;
}

/* Welcome text */
.navbar__user {
    font-size: 16px;
    font-weight: 500;
    opacity: 0.9;
}

/* Button */
.navbar__button {
    padding: 8px 18px;
    border-radius: 999px;
    border: none;
    cursor: pointer;
    font-weight: 600;
    transition: all 0.3s ease;
}

/* Sign In */
.navbar__button--primary {
    background-color: #3b82f6;
    color: white;
}

.navbar__button--primary:hover {
    background-color: #2563eb;
    transform: translateY(-2px);
}

/* Sign Out */
.navbar__button--danger {
    background-color: #ef4444;
    color: white;
}

.navbar__button--danger:hover {
  background-color: #dc2626;
  transform: translateY(-2px);
}

/* Responsive */
@media (max-width: 768px) {
  .navbar {
    padding: 0 20px;
  }

  .navbar__brand {
    font-size: 18px;
  }

  .navbar__user {
    display: none;
  }
}
```

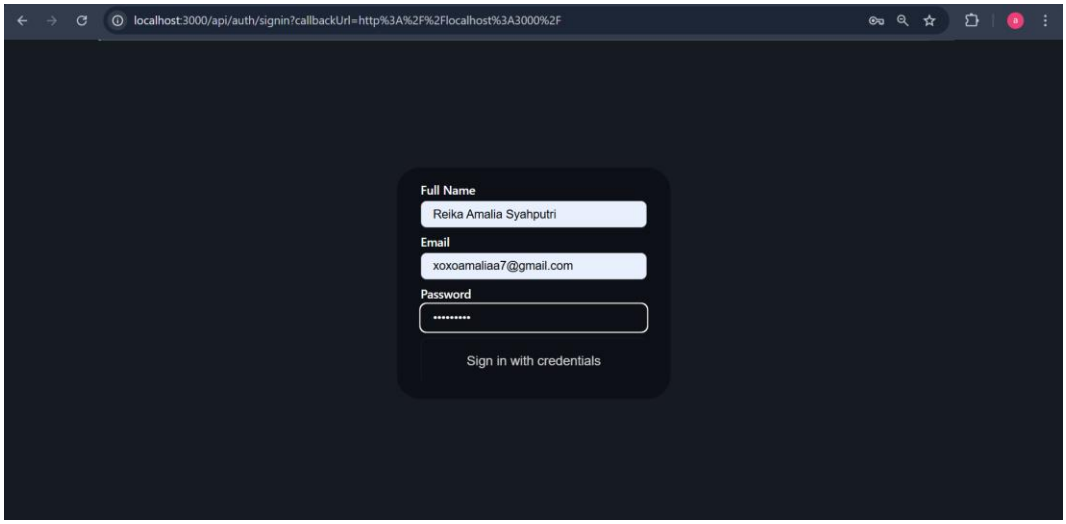
- Modifikasi index.tsx pada folder components/layouts/navbar

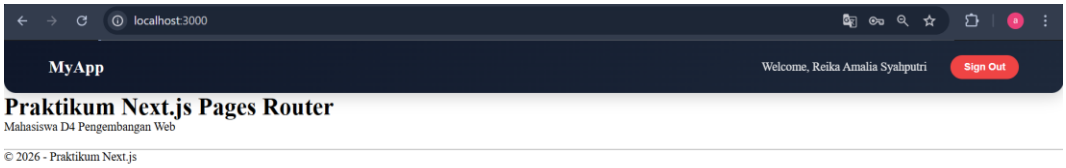
```tsx
import styles from './navbar.module.css';
import { signIn, signOut, useSession } from 'next-auth/react';

const Navbar = () => {
  const {data}: any = useSession()
  //const { data: session } = useSession()
  // console.log("session", session)
  return (
    <div className={styles.navbar}>
      <div className={styles.navbar__brand}>
        MyApp
      </div>

      <div className={styles.navbar__right}>
        {data ? (
          <>
            <div className={styles.navbar__user}>
              Welcome, {data.user?.fullname}
            </div>
            <button
              className={`${styles.navbar__button} ${styles["navbar__button--danger"]}`}
              onClick={() => signOut()}
            >
              Sign Out
            </button>
          </>
        ) : (
          <button
            className={`${styles.navbar__button} ${styles["navbar__button--primary"]}`}
            onClick={() => signIn()}
          >
            Sign In
          </button>
        )}
      </div>
    </div>
  );
};

export default Navbar;
```

- Jalankan browser pada localhost



- Lakukan sign in





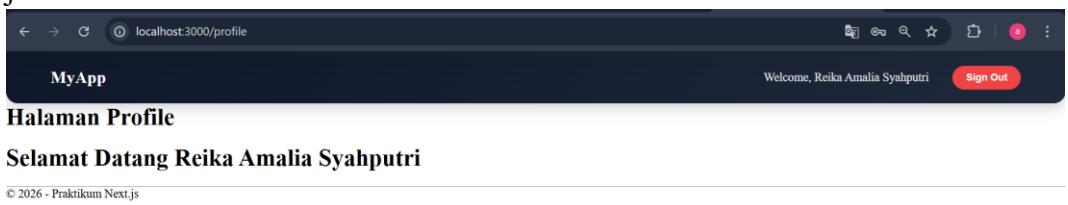
E. Proteksi Halaman Profile

Buat Halaman Profile

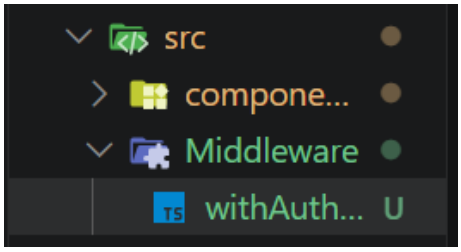
- pages/profile/index.tsx dan modifikasi file index.tsx

```
import { useSession } from 'next-auth/react';

const ProfilePage = () => {
  const { data }: any = useSession();
  return (
    <div>
      <h1>Halaman Profile</h1> <br />
      <h1>Selamat Datang {data?.user?.fullname}</h1>
    </div>
  );
};

export default ProfilePage;
```

- jalankan browser



- Buat Middleware Authorization
  Buat file withAuth.ts dan folder dengan nama middleware di src



- Modifikasi withAuth.ts

```
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });
      if (!token) {
        const loginUrl = new URL("/auth/login", req.url);
        return NextResponse.redirect(loginUrl);
      }
    }
    return middleware(req, next);
  };
}

```

- Modifikasi file middleware.ts

```
import { NextMiddleware, NextResponse } from "next/server";
import type { NextRequest } from "next/server";
import withAuth from "./Middleware/withAuth";

function mainMiddleware(request: NextRequest) {
  return NextResponse.next();
}

export default withAuth(mainMiddleware as NextMiddleware, ["/produk", "/about", "/profile"]);

export const config = {
  matcher: ["/produk", "/about", "/profile"],
};

```

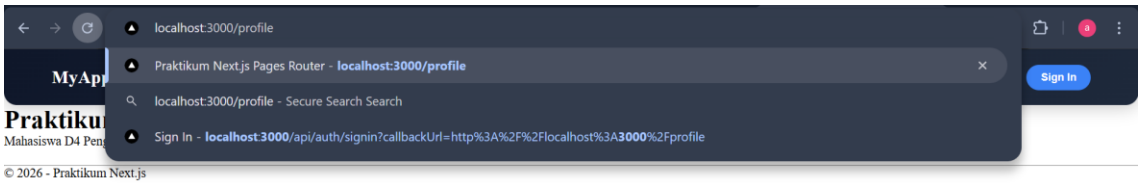
- Jika user mengarahkan ke halaman profile tidak akan bisa, user akan diarahkan ke alamat localhost

F. Pengujian

Uji 1 – Belum Login

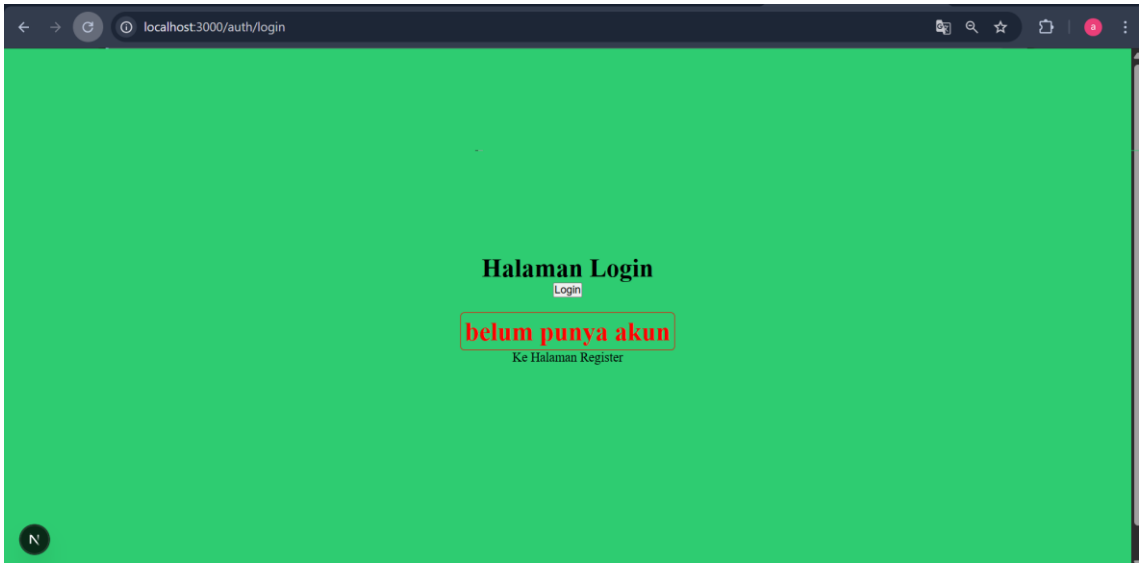
Akses:

/profile



Hasil:

Redirect ke home

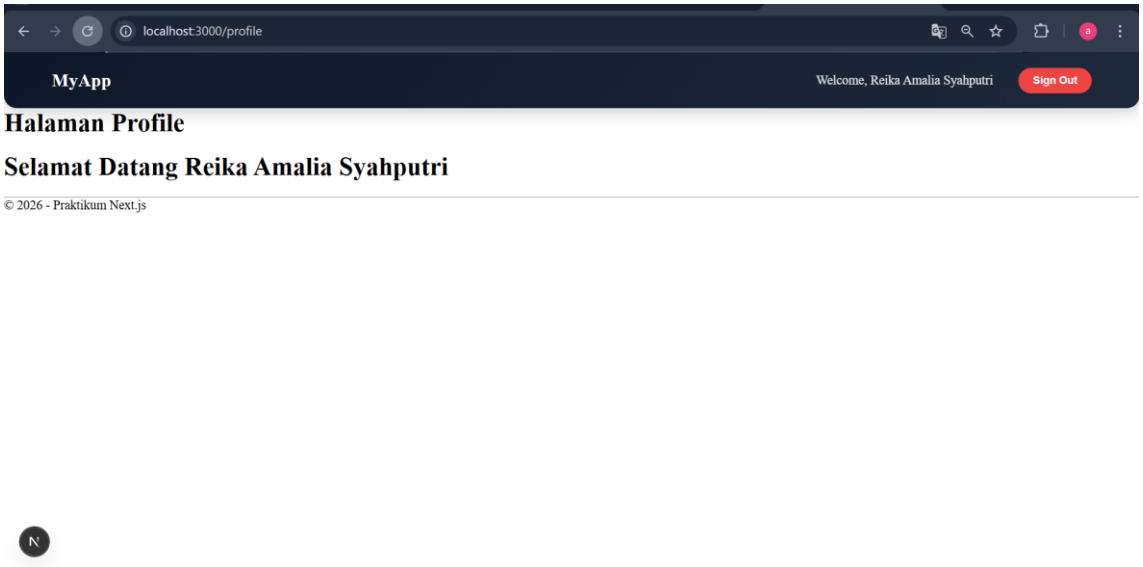


Uji 2 – Sudah Login

Login terlebih dahulu → Akses /profile

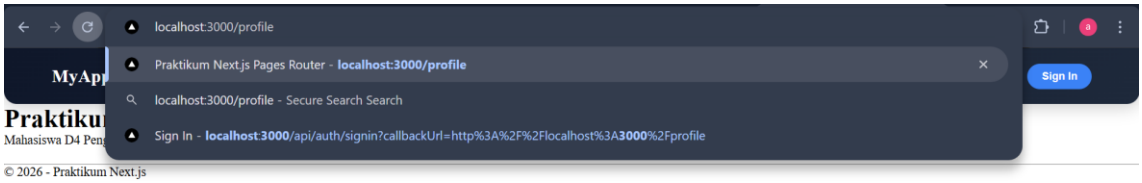
Hasil:

Bisa masuk



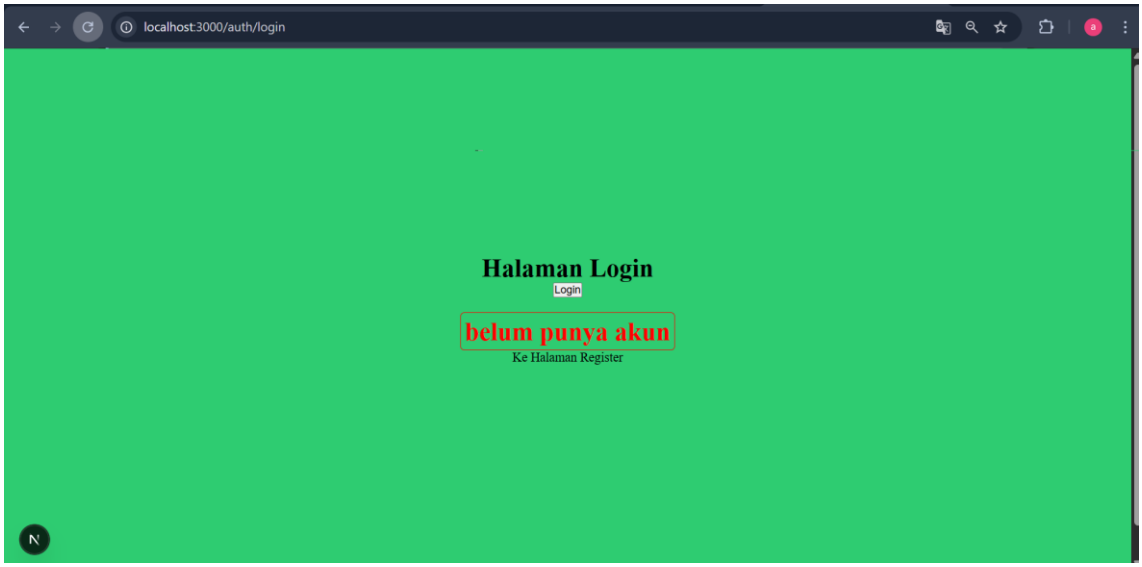
Uji 3 – Logout

Klik Sign Out → Akses /profile



Hasil:

Tidak bisa masuk



H. Tugas Praktikum

1. Implementasikan login menggunakan Credentials Provider.

```python
import NextAuth, { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt"
  },
  secret: process.env.NEXTAUTH_SECRET,

  providers: [
    // credentials provider untuk login
    CredentialsProvider({
      name: "credentials",
      credentials: {
        fullname: { label: "Full Name", type: "text" },
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        const user: any = {
          id: "1",
          email: credentials?.email,
          password: credentials?.password,
          fullname: credentials?.fullname, // field full name
        };
        if (user) {
          return user;
        } else {
          return null;
        }
      }
    })
  ],

  callbacks: {
    async redirect({ url, baseUrl }) {
      if (url.startsWith(baseUrl)) return url;
      if (url.startsWith("/")) return `${baseUrl}${url}`;
      return `${baseUrl}/profile`;
    },

    async jwt({ token, account, profile, user }: any) {
      if (account?.provider === "credentials" && user) {
        token.email = user.email;
        token.fullname = user.fullname;
      }
      return token;
    },
    async session({ session, token }: any) {
      if (token.email) {
        session.user.email = token.email;
      }
      if (token.fullname) {
        session.user.fullname = token.fullname;
      }
      return session;
    }
  }
};

export default NextAuth(authOptions);
```

2. Tambahkan field full name.

```python
import NextAuth, { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt"
  },
  secret: process.env.NEXTAUTH_SECRET,

  providers: [
    // credentials provider untuk login
    CredentialsProvider({
      name: "credentials",
      credentials: {
        fullname: { label: "Full Name", type: "text" },
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        const user: any = {
          id: "1",
          email: credentials?.email,
          password: credentials?.password,
          fullname: credentials?.fullname, // field full name
        };
        if (user) {
          return user;
        } else {
          return null;
        }
      }
    })
  ],

  callbacks: {
    async redirect({ url, baseUrl }) {
      if (url.startsWith(baseUrl)) return url;
      if (url.startsWith("/")) return `${baseUrl}${url}`;
      return `${baseUrl}/profile`;
    },

    async jwt({ token, account, profile, user }: any) {
      if (account?.provider === "credentials" && user) {
        token.email = user.email;
        token.fullname = user.fullname;
      }
      return token;
    },
    async session({ session, token }: any) {
      if (token.email) {
        session.user.email = token.email;
      }
      if (token.fullname) {
        session.user.fullname = token.fullname;
      }
      return session;
    }
  }
};

export default NextAuth(authOptions);

```

3. Tampilkan full name setelah login.
src/components/navbar/index.tsx

```jsx
<div className={styles.navbar__user}>
                Welcome, {data.user?.fullname} {/* show full name  */}
              </div>
```

src/pages/profile/index.tsx

```
import { useSession } from 'next-auth/react';

const ProfilePage = () => {
  const { data }: any = useSession();
  return (
    <div>
      <h1>Halaman Profile</h1> <br />
      <h1>Selamat Datang {data?.user?.fullname}</h1>
    </div>
  );
};

export default ProfilePage;

```

4. Buat halaman profile.


```
import { useSession } from 'next-auth/react';

const ProfilePage = () => {
  const { data }: any = useSession();
  return (
    <div>
      <h1>Halaman Profile</h1> <br />
      <h1>Selamat Datang {data?.user?.fullname}</h1>
    </div>
  );
};

export default ProfilePage;

```


5. Lindungi halaman profile dengan middleware.
src/middleware.ts

```
import { NextMiddleware, NextResponse } from "next/server";
import type { NextRequest } from "next/server";
import withAuth from "./Middleware/withAuth";

function mainMiddleware(request: NextRequest) {
  return NextResponse.next();
}

export default withAuth(mainMiddleware as NextMiddleware, ["/produk", "/about", "/profile"]);

export const config = {
  matcher: ["/produk", "/about", "/profile"],
};

```

src/Middleware/withAuth.ts

```ts
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });
      if (!token) {
        const loginUrl = new URL("/auth/login", req.url);
        return NextResponse.redirect(loginUrl);
        // perlindungan untuk halaman yang membutuhkan autentikasi, jika tidak ada token maka akan diarahkan ke halaman login menggunakan middleware
      }
    }
    return middleware(req, next);
  };
}

```

i. Pertanyaan Analisis

1. Mengapa session menggunakan JWT?
   - Session menggunakan JWT karena tidak perlu menyimpan data session di server. Data user disimpan dalam bentuk token yang dikirim ke client. Hal ini membuat sistem lebih ringan dan mudah di-scale, karena server tidak perlu menyimpan banyak data session.

2. Apa perbedaan authorize() dan callback jwt()?
   - Fungsi authorize() digunakan untuk memvalidasi data login dari user, seperti email dan password. Jika valid, maka user akan dianggap berhasil login. Sedangkan callback jwt() digunakan untuk mengatur data yang akan disimpan ke dalam token JWT, misalnya email atau fullname. Jadi, authorize() untuk proses login, sedangkan jwt() untuk menyimpan data ke token.

3. Mengapa middleware perlu getToken()?
   - Middleware membutuhkan getToken() untuk membaca data token JWT dari user. Dengan token tersebut, middleware bisa mengecek apakah user sudah login atau belum. Jika tidak ada token, maka user akan diarahkan ke halaman login.

4. Apa risiko jika NEXTAUTH_SECRET tidak digunakan?
   - Jika NEXTAUTH_SECRET tidak digunakan, maka token JWT tidak aman karena tidak memiliki kunci enkripsi. Hal ini bisa menyebabkan token mudah dimanipulasi atau dipalsukan oleh pihak lain, sehingga berisiko terhadap keamanan sistem.

5. Apa perbedaan autentikasi dan otorisasi dalam sistem ini?
   - Autentikasi adalah proses untuk memastikan identitas user, misalnya saat login menggunakan email dan password. Sedangkan otorisasi adalah proses untuk menentukan hak akses user setelah berhasil login, misalnya apakah user boleh mengakses halaman tertentu atau tidak.